

Maintenance Agent

4-phase automated maintenance pipeline: Dependabot · Sitemap · SEO/ADA · Build

Overview

The Maintenance Agent is a 4-phase sequential pipeline built with Microsoft AutoGen (autogen-agentchat). Each phase is handled by a specialised AssistantAgent that can read and patch source files, interact with the GitHub API, and build/verify the codebase. All agents share the same LLM client (Gemini 2.5 Flash primary, gpt-4o-mini fallback) and communicate through a RoundRobinGroupChat with MaxMessageTermination.

Schedule & Invocation

Trigger	GitHub Actions workflow (run-maintenance-agent.yml) or manual CLI
CLI	python -m meridian_agents.maintenance_agent
SDK	autogen-agentchat + autogen-ext[openai]
Primary model	gemini-2.5-flash (via GOOGLE_API_KEY / GEMINI_API_KEY)
Fallback model	gpt-4o-mini (OPENAI_API_KEY required)
Max turns	Phase 1: 100 · Phase 2: 80 · Phase 3: 100 · Phase 4: 60

Pipeline Phases & Subagents

Phase 1 — DependabotAgent

Lists all open GitHub PRs via list_github_prs(). For each Dependabot PR: LOW/MEDIUM risk → merge immediately; HIGH risk → reads changed files, applies code fixes if needed via branch/PR workflow, then merges the Dependabot PR. Skips non-Dependabot PRs entirely.

Tools: list_github_prs, get_pr_details, merge_github_pr, close_github_pr, get_github_repo, git_checkout_branch, git_pull_main, git_stage_and_commit, git_push_current_branch, create_github_pr, delete_github_branch, read_source_file, apply_file_patch, revert_file, install_frontend_deps, run_frontend_build, build_nodejs_service, check_python_syntax, git_status_short, git_diff_changes

Phase 2 — SitemapAgent

Reads or generates frontend/public/sitemap.xml. Checks every URL: for /post/:slug and /story/:id it queries the backing API. Fixes 404s via the branch/PR workflow. Regenerates sitemap after fixes.

Tools: get_sitemap_urls, generate_sitemap, check_url_status, + branch/PR tools + code tools

Phase 3 — SEOADAAgent

Runs 3 sequential audits: (1) `check_frontend_static()` — SEO/ADA checklist against source files; (2) `check_internal_links()` — scans Vue/JS files for broken RouterLink/href paths; (3) per-post SEO via `analyze_post_seo()` and ADA via `analyze_html_fragment()`. Applies all fixes via the branch/PR workflow.

Tools: `check_frontend_static`, `check_internal_links`, `fetch_all_posts`, `analyze_post_seo`, `fetch_post_html`, `analyze_html_fragment`, + `branch/PR` + `code` + `build tools`

Phase 4 — BuildVerifier

Pulls latest main, then runs `npm ci` + `npm run build` for each of the 5 Node.js services. Also runs Python syntax check across all `.py` files. Fixes any build failures via branch/PR. Loops up to 3 times per failing service.

Tools: `git_pull_main`, `build_nodejs_service`, `check_python_syntax`, `install_frontend_deps`, `run_frontend_build`, + `branch/PR` + `code tools`

Branch / PR Workflow (shared by all fix agents)

All code fixes follow a mandatory 10-step workflow: (1) `get_github_repo`, (2) `git_checkout_branch` from main, (3) `apply_file_patch` (read first), (4) `run_frontend_build` for Vue changes, (5) `git_stage_and_commit`, (6) `git_push_current_branch`, (7) `create_github_pr`, (8) `merge_github_pr` squash, (9) `delete_github_branch`, (10) `git_pull_main`. If `GITHUB_REPO` is not set, fixes are applied directly on main with no PR.

Tools Reference

Tool	Purpose
<code>check_frontend_static()</code>	Audits <code>index.html</code> , <code>robots.txt</code> , router, <code>App.vue</code> , <code>Navbar</code> , <code>Footer</code> , <code>BlogPost.vue</code> for SEO/ADA
<code>check_internal_links()</code>	Scans Vue/JS files for RouterLink/href paths that don't match any router route
<code>analyze_post_seo(title, excerpt, slug, wc)</code>	Scores a post on title length, excerpt length, slug format, word count
<code>analyze_html_fragment(html, ctx)</code>	WCAG AA scan: missing alt, unlabelled buttons, broken link text, heading skips, tables
<code>build_nodejs_service(service)</code>	<code>npm ci</code> + <code>npm run build</code> for one of the 5 services; catches peer-dep conflicts
<code>check_python_syntax()</code>	<code>py_compile</code> scan of all <code>.py</code> files under <code>meridian_agents/</code>
<code>generate_sitemap()</code>	Reads Vue router + live API to build <code>sitemap.xml</code> with static + dynamic routes
<code>list_github_prs()</code> / <code>get_pr_details()</code>	GitHub REST API: list open PRs, get semver bump and risk level
<code>merge_github_pr()</code> / <code>close_github_pr()</code>	GitHub REST API: squash-merge or close a PR
<code>apply_file_patch(path, old, new)</code>	Surgical text replacement — <code>old_string</code> must appear exactly once

Tool	Purpose
revert_file(path)	git checkout HEAD -- to undo a failed patch

Tracer

tracer.py creates an AgentRun record at run start (POST /api/agent-runs) and updates it on completion/failure (PUT /api/agent-runs/). The record stores the run UUID, start/end timestamps, summary, and structured findings.

Key Files

File	Purpose
maintenance_agent/main.py	Entry point, environment validation, run_id management
maintenance_agent/agents.py	4-phase pipeline, all AssistantAgent definitions, _run_agent helper
maintenance_agent/tools.py	All ~30 tool functions: audit, patch, git, GitHub API, build, sitemap
maintenance_agent/tracer.py	AgentRun lifecycle tracking via Meridian API